

Nome e Cognome _____ Matricola _____

Corso di Calcolo Parallelo

Laurea Magistrale in Informatica—Università di Bologna

Prova scritta, 10 Giugno 2026

- La prova dura 90 minuti
- Durante la prova non è consentito consultare libri, appunti o altro materiale.
- Non è consentito l'uso di dispositivi elettronici (ad esempio, cellulari, tablet, smart watch/smart glass, ...), né interagire in alcun modo con gli altri ammessi alla prova pena l'esclusione, che potrà avvenire anche dopo il termine della stessa.
- Le risposte devono essere scritte **a penna** su questi fogli, in modo **leggibile**. Le parti illeggibili o scritte a matita saranno ignorate.
- Eventuali altri fogli possono essere utilizzati per la brutta copia ma non verranno valutati.
- Per ritirarsi è sufficiente scrivere "RITIRATO" sulla prima pagina di questo plico.
- I voti saranno pubblicati su AlmaEsami.
- I voti della prova scritta e del progetto restano validi fino alla sessione d'esame di **febbraio 2027** inclusa. Dopo tale data tutti i voti in sospeso saranno persi.

NON SCRIVERE NELLA TABELLA SOTTOSTANTE

D. 1	D. 2	D. 3	D. 4	D. 5
/ 6	/ 6	/ 6	/ 6	/ 6

Domanda 0.

Scrivere nome, cognome e numero di matricola in cima al primo foglio.

Domanda 1.

Discutere le ragioni per le quali, nella pratica, si osserva raramente uno speedup ideale.

Risposta.

Domanda 2.

Discutere se e come sia possibile parallelizzare il frammento di codice seguente aggiungendo opportuni costrutti OpenMP, senza stravolgere il programma. Solo nel caso in cui il codice possa essere parallelizzato, discutere le eventuali limitazioni della soluzione proposta.

Assumere che $\text{fun}(i, j)$ restituisca un valore che dipende solo dai valori di i e j , e non dal contenuto dell'array $F[]$.

```
1  #define N ...
2  float F[N];
3
4  /* Inizializzazione di F[N] non riportata */
5
6  for (i=0; i<N; i++) {
7      for (j=i+1; j<N; j++) {
8          const float f = fun(i, j);
9          F[i] += f;
10         F[j] += f;
11     }
12 }
```

Risposta.

Domanda 3.

Descrivere un programma MPI per calcolare il prodotto $c[N]$ tra una matrice quadrata $A[N][N]$ e un vettore $b[N]$, entrambi di tipo *float*. Ricordiamo che è possibile calcolare il prodotto matrice-vettore con il seguente frammento di codice seriale:

```
1  for (int i=0; i<N; i++) {  
2      c[i] = 0;  
3      for (int j=0; j<N; j++)  
4          c[i] += A[i][j] * b[j];  
5  }
```

Si assuma che:

- La dimensione N della matrice A e del vettore b siano inizialmente noti a tutti i processi;
- N sia un multiplo del numero di processi MPI;
- Il contenuto di $A[][]$ e $b[]$ siano inizialmente noti solo al processo 0;
- Al termine della computazione, il risultato $c[]$ debba essere noto al processo 0.

Risposta.

Domanda 4.

1. Illustrare brevemente i principali tipi di memoria presenti in una GPU CUDA-
2. Perché c'è una gerarchia così complessa? Non sarebbe stata sufficiente solo la memoria globale? Discutere.

Risposta.

Domanda 5.

La funzione seguente, vista a lezione, calcola la somma del contenuto di un array $v[]$ di lunghezza n senza usare la clausola `reduction()`.

```
1 long arraysum(const int *v, int n)
2 {
3     const int P = omp_get_max_threads();
4     long local_sum[P];
5     long sum = 0;
6     #pragma omp parallel
7     {
8         const int my_id = omp_get_thread_num();
9         const int my_start = (n * my_id) / P;
10        const int my_end = (n * (my_id+1)) / P;
11        local_sum[my_id] = 0;
12        for (int i=my_start; i<my_end; i++)
13            local_sum[my_id] += v[i];
14    }
15    for (int k=0; k<P; k++)
16        sum += local_sum[k];
17    return sum;
18 }
```

Assumere che il *pragma* alla riga 6 crei P thread OpenMP, dove P è il valore calcolato alla riga 3.

1. Determinare *work*, *span* e *speedup* della funzione `arraysum()` in funzione della lunghezza n dell'array e del numero P di unità di esecuzione; assumere che sia n che P possano variare arbitrariamente, quindi le espressioni calcolate potranno dipendere sia da n che da P . Motivare la risposta.
2. Discutere lo *speedup* nel caso (ipotetico) in cui $P = n$. Cosa succede se n tende all'infinito?
3. Discutere lo *speedup* nel caso (più realistico) in cui P sia una costante "piccola". Cosa succede in questo caso se n tende all'infinito? (*Suggerimento: per calcolare il limite per x che tende a infinito di un rapporto del tipo $\frac{ax+b}{cx+d}$ si può raccogliere l'incognita ottenendo $\frac{x(a+b/x)}{x(c+d/x)}$ e quindi semplificare x .*)

Risposta.